Samarth Puri **CMPSC 448- Midterm Project (Individual)**

Notes: LSTM is what I used for my RNN. I do not wish to present.

Problem Definition & Data Curation:

In this problem, we hope to discover how accurate the CNN and RNN models are in identifying which LLMs'(ChatGPT, Claude, Deepseek) generated a particular response given a specific prompt. In my dataset, each LLM is given the same exact 300 questions (inputs) totaling to 900 total questions. My dataset originally only included 300 total inputs but due to low model performances, I decided to length the dataset by 3x so there would be more examples the model could learn from. The first 100 input prompts are from a free website (listed below) and the other 200 are generated through AI interactions.

CNN/RNN Implementation & Training:

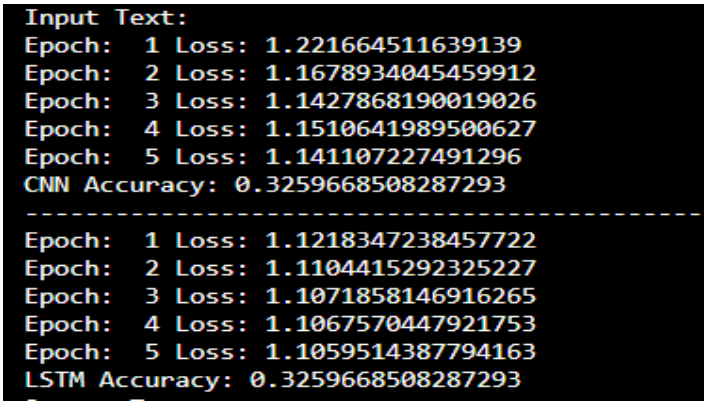
I began by importing the dataset and tokenizing the inputs and outputs into numbers that the model can recognize. I did this by splitting each input/output into individual tokens and replacing each token with an assigned, unique number. Then, I grouped these numbers that represented 80% training and 20% testing and mapped each set of numbers to their respective LLM identity label.

Using torch packages, I designed both CNN and LSTM (RNN) models that followed similar steps. For the training, I assigned a learning rate and ran a for loop that calculates loss by comparing the predictions and the correct answers and uses that to update its gradients through backpropagation and learn correct answers. Then for testing, the training model would use a separate, unread portion of the dataset and would make predictions. The number of correct predictions is incremented by 1 and is divided over the total Y_test_batch to calculate total accuracy.

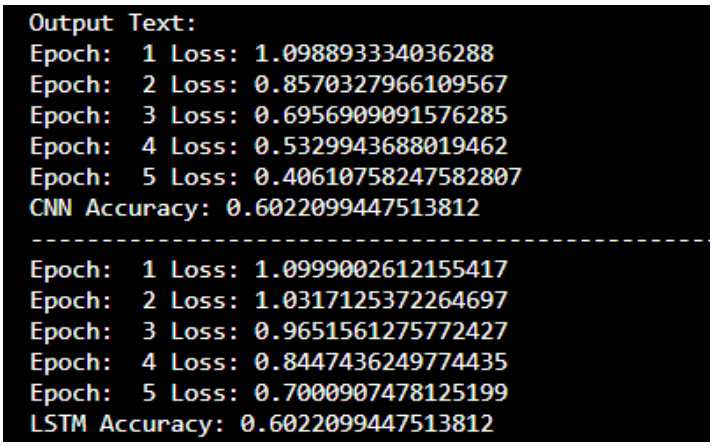
For the CNN model, I used an embedding layer that converts each token into a vector of 100 dimensions. I then used a convolutional layer with 128 filters and a kernel size of 3 which analyzes 3 sequential tokens each time to identify patterns between consecutive words. After that, I applied ReLU activation and max pooling before passing the results through a fully connected layer to predict the LLM. As for the LSTM model, I also used a 100-dimensional embedding layer, followed by an LSTM layer with a hidden size of 128 to learn sequential patterns between words. I then applied max pooling and a fully connected layer to predict the LLM. For both models, I used CrossEntropyLoss as the loss function and the Adam optimizer with a learning rate of 0.001. I trained both models for 5 epochs using a batch size of 32.

Results:

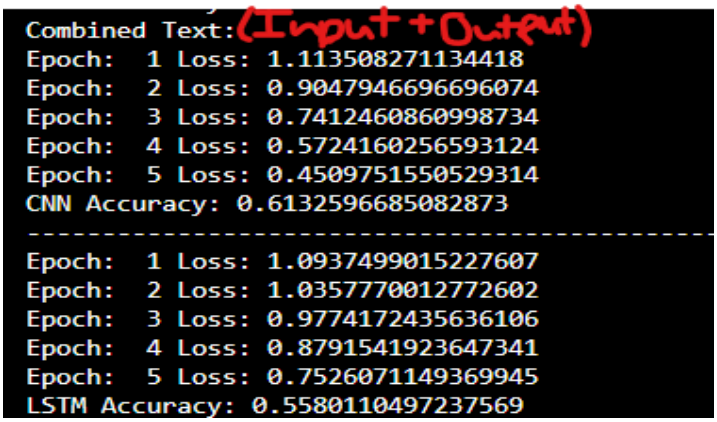
Input Only:



Output Only:



Input & Output Only:



These pictures highlight the epochs, their corresponding losses, and CNN and LSTM accuracy for input only, output only and input & output.

Comparing the three experiments, input-only classification performed close to random guessing at about 33% which was expected because each LLM received the same prompts. However, output only classification displayed an increase in accuracy for both CNN and LSTM hinting that the generated responses have distinctive patterns. When combining the input and output, CNN accuracy improved slightly compared to output-only classification, while LSTM accuracy had decreased. This suggests that including the prompt does not necessarily improve classification performance for both models.

In-Depth Analyses/Experiments:

Initially, after my first few runs, I happened to discover that I kept getting a lower accuracy percentage for both models (around 33%) during my "only input" runs compared to the other conditional runs. This led me to question and eventually become skeptical of my dataset. Since each LLM in my dataset received the same exact prompts, there was no uniqueness within the prompt that allowed to model to identify its associated LLM. For instance, if each LLM is given the prompt "Give a metaphor for the scientific method", there is no distinctive identifier/property for a model to guess its LLM. The accuracy percentage of 32.60% (about 1/3) confirms this idea due to the format of the dataset, each input question is associated equally with all three LLM labels. (ChatGPT, Claude, and DeepSeek) Therefore, based on the input alone, the model has a 1/3 chance of selecting the correct LLM. Since this is true across all questions, the expected overall accuracy is approximately 1/3, or 33.33%.

However, the other runs (output only and input & output) were higher in accuracy percentages for both LLMs. This would make sense because each output was generated by different LLM and hence held different identifiers making it easier to distinguish. All these runs had accuracy percentages of over 50%, meaning that majority of the time the model was able to correctly identify the prompt's LLM suggesting that each generated output had different, unique writing styles and features.

Lessons & Experiences Learned:

Initially, I had planned and included only 300 prompts and their associated LLMs in my dataset so each LLM was split evenly with 100 examples. After I used this in my dataset, however, both models' accuracy was too low leading me to highly suspect my dataset. After enlarging my dataset by 3x, my accuracy increased because my models had learned from more examples and thus, more identifiers.

Another thing I learned is the way I split the data that was special to my case because of my dataset structure. I wanted to split the dataset into 80% training and 20% testing. However, because my dataset repeated one prompt three times (one for each LLM), the same prompt can

appear in both training and testing leading to incorrect predictions. That's why I used .unqiue() to ensure that the models can learn specific patterns/characteristics because the same question won't appear in both training and testing. Instead, the CNN and LSTM will be fed unseen prompts to providing a better measure of how well they could generalize to new questions.\

Bonus:

RQ4:

LLM_name	response_length	vocabulary_diversity
ChatGPT	43.103333	0.875397
Claude	63.453333	0.833326
Deepseek	35.490000	0.892632

Before word reduction:

```
Output Text:
Epoch:  1 Loss: 1.098893334036288
Epoch:  2 Loss: 0.8570327966109567
Epoch:  3 Loss: 0.6956909091576285
Epoch:  4 Loss: 0.5329943688019462
Epoch:  5 Loss: 0.40610758247582807
CNN Accuracy: 0.6022099447513812
-------------------------------------------------
Epoch:  1 Loss: 1.0999002612155417
Epoch:  2 Loss: 1.0317125372264697
Epoch:  3 Loss: 0.9651561275772427
Epoch:  4 Loss: 0.8447436249774435
Epoch:  5 Loss: 0.7000907478125199
LSTM Accuracy: 0.6022099447513812
```

After word reduction:

```
Shortened Responses:
Epoch:  1 Loss: 1.1362635830174321
Epoch:  2 Loss: 0.86917993037597
Epoch:  3 Loss: 0.7018656626991604
Epoch:  4 Loss: 0.5554813768552698
Epoch:  5 Loss: 0.4337103703747625
CNN Accuracy: 0.5193370165745856
-------------------------------------------------
Epoch:  1 Loss: 1.10287460555201
Epoch:  2 Loss: 1.0455425252085147
Epoch:  3 Loss: 0.989238687183546
Epoch:  4 Loss: 0.9153137440266816
Epoch:  5 Loss: 0.7977700674015543
LSTM Accuracy: 0.4972375690607735
```

The two linguistic and structural characteristics I tested were response length and vocabulary. I reduced the response output length to only include the first 30 words and it produced a lower accuracy for CNN and LSTM. Specifically, CNN accuracy decreased from 60.22% to 51.93%, while LSTM accuracy decreased from 60.22% to 49.72%. This suggests that both models benefit from having longer responses, as shortening the responses removes vocabulary and word patterns that may help distinguish between LLMs.

Also, there is a significant difference in response length between specific LLMs. For instance, on average, Claude generates a response length of almost 2x DeepSeek's response length. Hence, response length can reveal defining patterns and traits that can be used to predict LLM response characteristics. However, in regards, to vocabulary diversity which is calculated by taking the number of unique words over the number of total words, there seems to be little difference. For the all LLMs, they seem to average about 80-90 % of the response is unique.